Revisiting the Compact Tail Assignment Model: Corrections and an Exact Event-Based Maintenance Reformulation

Viktor Borodin[a], Arun Kumar Pappala[b]

[a] *Taras Shevchenko National University, Kyiv, Ukraine*
[b] *Singapore University of Technology and Design, Singapore*

Abstract

This paper revisits the compact mixed-integer linear programming model for the Tail Assignment Problem (TAP) introduced by Khaled et al. (2018). We make three contributions. First, we show that the basic continuity and turn-time inequalities (C2–C4; constraints (3)–(6) in Khaled et al.) admit a simultaneous-departure corner case that is mathematically feasible but physically infeasible, and we close this gap with explicit non-overlap constraints (C5). Second, we identify and formally correct a flaw in the paper's cumulative-flight-time row (constraint (13) in Khaled et al.; not our aggregation constraint C11): the original formulation allows maintenance intervals to be incorrectly relaxed. We show that a two-row endpoint split is also insufficient and give an exact cumulative-state formulation. We give a numeric counterexample, evaluated against the real built constraint objects at two adversarial check patterns, in which the original and endpoint-split formulations each fail on a different pattern while the ordered formulation rejects both. Third, we propose an alternative to Khaled et al.'s set of constraints for the same TAP problem with type-A maintenance: an ordered, day-free

event-based formulation that uses aircraft-local prefix state without a day-pair window. We prove the reformulation is correct and evaluate it alongside the two legacy rows and the strict corrected legacy variant. On the corrected feasible family, we compare five formulations on six deterministic A-only instances with zero-day legacy deferral. The strengthened legacy and event formulations match objectives on the four cells where all three prove optimality; four larger event-based runs reach the time limit. Independent replay accepts all optimal strengthened-legacy and event schedules, while the original and endpoint-split C13 schedules violate the flight-hour threshold. The experiments do not establish a general timing advantage. All code, instances, and result tables accompany the project.

Keywords: Tail assignment, Aircraft routing, Maintenance scheduling, Integer programming, Mixed-integer programming corrections

---

1. Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning (Barnhart et al., 2003; Gopalan and Talluri, 1998). The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, suboptimal maintenance routing, and safety-margin violations.

In addition to the basic assignment and continuity requirements, the TAP

must respect aircraft maintenance constraints. The most common maintenance type is *type-A*, which requires a maintenance check after a maximum flying-time interval. Type-A maintenance is typically performed at a limited set of airports, and the TAP must ensure that each aircraft is routed to a maintenance-capable airport before its flying-time limit is exceeded. The type-A maintenance requirement is a key operational constraint in the TAP and has been studied extensively in the literature.

Khaled et al. (2018) proposed a compact 0–1 linear programming formulation for the TAP without maintenance, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of Levin (1971), which requires $O(n_f^2 n_p)$ variables. Further, Khaled et al. (2018) extended the model to include type-A maintenance requirements, which are defined by a maximum flying-time interval between maintenance checks, that adds other $O(n_f n_p D)$ binary variables, where D is the number of calendar days in the planning horizon. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

Our contributions

The present paper makes three contributions that check and correct those results:

1. **Basic feasibility correction (C1–C4 / Khaled constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one

aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counter-example and enforce physical executability by adding explicit pairwise non-overlap constraints (our C5).

Our non-overlap correction retains the $O(n_f n_p)$ binary-variable space but can add $O(n_f^2 n_p)$ conflict inequalities in the worst case. In practical timetables, only temporally conflicting pairs are instantiated; the resulting constraint count is therefore governed by the sparse conflict set $\mathcal{O}$ defined in Section 3.

2. **Cumulative-flight-time row correction.** We show that the cumulative-flight- time maintenance constraint (constraint (13) in Khaled et al. (2018)) contains a logical flaw in opening and one-endpoint intervals, and in the handling of consecutive flights and maintenance events. We also show by counterexample that a natural two-row endpoint split does not fully repair the model. We therefore formulate an exact cumulative-state recursion that propagates and resets the flight-hour counter without day-pair ambiguity.
3. **Event-based reformulation.** The day-indexed type-A model above still repeats a calendar-day index across every maintenance variable. We give a complete, day-free reformulation of the same type-A C1–C13 requirements, in which each flight's own timestamp supplies all calendar information and maintenance is represented as a virtual flight triggered by, and indexed on, the preceding real flight rather than by calendar day. This removes the day dimension from the maintenance model's size and lets the flight-hour counter propagate exactly along each aircraft's

actual route, with no day-pair window and therefore no possibility of the split-form failure mode proved for the legacy cumulative-flight-time row.

Paper organisation

Section 2 reviews related work. Section 3 restates the basic TAP model and proves a C1–C4 corner-case correction closed by C1–C5. Section 4 presents the maintenance-extended model, the cumulative-flight-time row correction, and the induced interpretation updates for C8–C13 under the corrected basic model. Section 5 gives the day-free event-based reformulation of the type-A requirements and proves its correctness directly. Section 6 reports the numeric cumulative-flight-time counterexample, the five-method A-only benchmark, and its three-seed subset. Section 7 concludes.

2. Literature Review

2.1. Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation.. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required (Lavoie et al., 1988; Minoux, 1984; Barnhart et al., 1998; Sarac et al., 2006). Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in Borndörfer et al. (2010) and Froyland et al. (2013).

Multi-commodity flow.. Levin (1971) introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in Khaled et al. (2018).

Time-space networks.. Hane et al. (1995) proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and overnight stays. Extensions to aircraft routing appear in Clarke et al. (1996). Preprocessing by node aggregation and island isolation reduces model size (Hane et al., 1995; Sherali et al., 2006). Daily and weekly maintenance routing variants are studied in Liang et al. (2011) and Liang and Chaovalitwongse (2012); the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations.. Haouari et al. (2012) present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model of Khaled et al. (2018).

2.2. MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of Khaled et al. (2018), several mixed-integer alternatives are relevant for exact TAP modelling and large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight–

aircraft assignment pairs, often yielding stronger network-structure constraints (Levin, 1971; Hane et al., 1995; Sherali et al., 2006).

- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-and-price; this is exact but algorithmically heavier (Barnhart et al., 1998; Sarac et al., 2006; Klabjan, 2005).

- **Decomposed MILP frameworks:** Benders- or Dantzig–Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets (Cordeau et al., 2001; Mercier et al., 2005; Desaulniers et al., 1997).

- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime (Clarke et al., 1996; Liang et al., 2011; Liang and Chaovalitwongse, 2012).

- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style linking constraints, producing schedules that are less brittle operationally (Borndörfer et al., 2010; Froyland et al., 2013).

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the alternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.3. Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed, and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this paper uses Khaled et al. (2018) as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.4. Maintenance constraints

Maintenance scheduling within aircraft routing has been studied since the early work of Clarke et al. (1997). The column-generation framework of Barnhart et al. (1998) can embed type-A constraints through constrained shortest-path pricing. Cordeau et al. (2001) apply Benders decomposition to simultaneous routing and crew scheduling with non-linear maintenance resources. Lacasse-Guay et al. (2010) consider maintenance under different business-process models. We focus on the type-A flight-hour case and its exact event-based reformulation.

2.5. Heuristic approaches

Constructive heuristics for TAP are comparatively under-studied relative to exact methods. Grönkvist (2005) provides a comprehensive treatment including local-search improvements. Heuristic and metaheuristic solution

methods are outside the scope of the present exact-formulation study and are not evaluated here.

2.6. Position of the present paper

The compact formulation of Khaled et al. (2018) represents the reference compact baseline for exact, cyclic-constraint-free TAP in this study. The present work builds directly upon it, providing: (i) a diagnosis of the C1–C13 formulation in Khaled et al. (2018) and an exact state-based replacement for the legacy cumulative-flight-time row, (ii) a new reformulation of the event-based model that resolves the counterexamples and improves computational performance, and (iii) an instance-matched benchmark of the implemented exact formulations.

3. The Basic Tail Assignment Model

We restate the compact model of Khaled et al. (2018) using the notation of Table 1. This section covers the model without maintenance constraints; they are introduced in Section 4.

3.1. Decision variables and objective

Let us define the decision variables and constraints for the basic tail assignment model.

C1 — Binary assignment domain.. The decision variable x_{ij} indicates whether flight i is assigned to aircraft j.

$$x_{ij} = \begin{cases} 1 & \text{if flight } i \text{ is operated by aircraft } j, \\ 0 & \text{otherwise,} \end{cases} \qquad i \in \mathcal{F},\ j \in \mathcal{P}. \quad (1)$$

Table 1: Notation summary.

Symbol	Meaning
$\mathcal{F} = \{1, \ldots, n_f\}$	Set of flight legs
$\mathcal{P} = \{1, \ldots, n_p\}$	Set of aircraft (tail numbers)
$\mathcal{A} = \{1, \ldots, n_k\}$	Set of airports
$a_i^{\uparrow}$, $a_i^{\downarrow}$	Departure / arrival airport of flight i
$t_i^{\uparrow}$, $t_i^{\downarrow}$	Departure / arrival time of flight i (minutes)
d_i, $\bar{d}_i$	Departure / arrival day derived from flight times
Δt	Minimum ground turn time (minutes)
c_{ij}	Assignment cost of flight i to aircraft j
a_j^0	Initial position airport of aircraft j
$F_k^{\downarrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\downarrow} = k,\ t_i^{\downarrow} \leq \theta - \Delta t\}$
$F_k^{\uparrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\uparrow} = k,\ t_i^{\uparrow} < \theta\}$

The objective minimises total assignment cost:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij}. \tag{2}$$

3.2. Constraints

C2 — Flight coverage.. Each flight must be assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \qquad \forall i \in \mathcal{F}. \tag{3}$$

C3,C4 — Equipment continuity (home/non-home airports).. For any aircraft j, any non-home airport $k \neq a_j^0$, and any flight i departing from k, aircraft j can be assigned to flight i only if it has already landed at k early enough,

but at the home airport ($k = a_j^0$), one free unit is available at time zero

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \;\geq\; x_{ij}, \qquad \forall j \in \mathcal{P},\; k \in \mathcal{A} \setminus \{a_j^0\},\; i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{4}$$

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \;\geq\; x_{ij} - 1, \qquad \forall j \in \mathcal{P},\; k = a_j^0,\; i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{5}$$

3.3. Validity

Constraints (4)–(5) are intended to encode both equipment continuity (C2 of Khaled et al. (2018)) and turn-time feasibility (constraint (6) of that paper). The set $F_k^{\downarrow}(t_i^{\uparrow})$ counts only arrivals at least Δt minutes before the departure of flight i, so a prior arrival can supply flight i only after the required turn time. This mechanism is valid when departures are strictly ordered, but it does not prevent two equal-time departures from using the same available aircraft, as shown below.

Proposition 1 (Claimed in Khaled et al. 2018)**.** *Any feasible solution to* (1)–(5) *corresponds to a feasible flight plan satisfying the original continuity and turn-time conditions.*

The proposition is not valid as stated when distinct flights may have equal departure times. The following counterexample identifies the missing case.

3.4. Counter-example with simultaneous departures

The balance constraints (4)–(5) use $F_k^{\uparrow}(\theta) = \{i \in \mathcal{F} : a_i^{\uparrow} = k,\; t_i^{\uparrow} < \theta\}$ with a strict inequality in departure time. If two flights leave the same

airport at the same time, neither flight appears in the other's departure-balance term. This can make an assignment feasible while being physically impossible as a flight plan.

Example 1 (Feasible to (1)–(5), infeasible in practice). Consider one aircraft j with initial airport $a_j^0 = \mathrm{B}$ and turn time $\Delta t = 30$ minutes. Let the three flights be:

$$\begin{aligned} i_0 &: \mathrm{B} \to \mathrm{A}, \quad t^{\uparrow}_{i_0} = 500,\ t^{\downarrow}_{i_0} = 550, \\ i_1 &: \mathrm{A} \to \mathrm{B}, \quad t^{\uparrow}_{i_1} = 600,\ t^{\downarrow}_{i_1} = 700, \\ i_2 &: \mathrm{A} \to \mathrm{C}, \quad t^{\uparrow}_{i_2} = 600,\ t^{\downarrow}_{i_2} = 700. \end{aligned}$$

Set $x_{i_0 j} = x_{i_1 j} = x_{i_2 j} = 1$.

Coverage (3) is satisfied (single aircraft, all flights assigned). Flight i_0 satisfies the home-airport constraint (5), because no flight has arrived at or departed from B before time 500. For i_1 at the non-home airport A:

$$F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \{i_0\}, \qquad F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \varnothing,$$

because i_0 departs from B and $t^{\uparrow}_{i_2} = 600 \not< 600$. Hence constraint (4) gives

$$\sum_{i' \in F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} - \sum_{i' \in F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} = 1 - 0 = 1 \geq x_{i_1 j} = 1.$$

Symmetrically for i_2, $t^{\uparrow}_{i_1} = 600 \not< 600$, so the same calculation gives $1 \geq x_{i_2 j} = 1$. Therefore (1)–(5) are feasible.

However, the resulting plan is physically infeasible: one aircraft cannot operate both i_1 and i_2 simultaneously at time 600.

Implication.. In implementations, this corner case is eliminated by adding pairwise non-overlap constraints for time-overlapping flights on the same aircraft, $x_{i_1 j} + x_{i_2 j} \leq 1$.

Corrected C1–C5 feasibility criterion.. For each flight i, define its operating interval, including the required turn-time buffer, by

Conflict-set definition..

$$I_i = [t_i^{\uparrow}, t_i^{\downarrow} + \Delta t). \tag{6}$$

The undirected conflict graph is $G = (\mathcal{F}, \mathcal{O})$, where its edge set is

$$\mathcal{O} = \{\{i, \ell\} \subseteq \mathcal{F} : i \neq \ell,\ I_i \cap I_\ell \neq \varnothing\}, \tag{7}$$

and $\mathcal{O}_i = \{\ell \in \mathcal{F} : \{i, \ell\} \in \mathcal{O}\}$ is the set of flights conflicting with i. For every conflict edge and aircraft, impose

$$x_{ij} + x_{\ell j} \leq 1, \qquad \forall \{i, \ell\} \in \mathcal{O},\ \forall j \in \mathcal{P}. \tag{8}$$

The implementation can strengthen these pairwise inequalities by using maximal cliques of G. Let $\mathcal{K}$ be the set of all maximal cliques, defined by

$$\mathcal{K} = \{K \subseteq \mathcal{F} : |K| \geq 2,\ \{i, \ell\} \in \mathcal{O}\ \forall i \neq \ell \in K,\ \text{and no strict superset of } K \text{ has this property}\}. \tag{9}$$

For each maximal clique, impose the C5 inequality

$$\sum_{i \in K} x_{ij} \leq 1, \qquad \forall K \in \mathcal{K},\ \forall j \in \mathcal{P}. \tag{10}$$

Every conflict edge is contained in a maximal clique, so the clique inequalities imply the pairwise inequalities for binary x. Conversely, a set of flights satisfying all pairwise inequalities contains at most one flight from each clique. Thus the two forms have the same integer feasible assignments, while the clique form can provide a stronger linear relaxation.

The corrected core model is therefore (1)–(10). This preserves the original $n_f n_p$ binary-variable space while restoring physical executability. It does not, in general, preserve the linear constraint count: the overlap family contains $|\mathcal{O}|n_p$ inequalities.

3.5. Model size

Proposition 2 (Analogue of Khaled et al. 2018, Proposition 2)**.** *The original model* (2)–(5) *with binary x contains $n_f n_p$ binary decision variables and $O(n_f n_p)$ constraints, while the corrected model has $n_f n_p$ binary variables and $O(n_f n_p + |\mathcal{O}|n_p)$ constraints.*

The corrected model retains $n_f n_p$ binary variables and has $O(n_f n_p + |\mathcal{O}|n_p)$ constraints. In the worst case $|\mathcal{O}| = O(n_f^2)$, although timetable sparsity normally makes the conflict set much smaller. The original variable count compares favourably with Levin's $O(n_f^2 n_p)$-variable model; Khaled et al. (2018) also report that the time-space network formulation uses approximately 1.25–2 times as many variables and constraints on their test instances.

3.6. Corrected proposition and its proof.

The issue with the proof of feasibility in the original model is that it does not account for simultaneous departures of overlapping flights. Moreover, the logic of the published proof goes in the wrong direction, and instead of proving that any solution of C1–C4 defines correct feasibility, it proves that any feasible assignment flights satisfy C1-C4.

The corrected proposition follows by considering the flights assigned to each aircraft in chronological order.

Proposition 3 (Proposition 1)**.** *Any feasible solution to* (1)–(10) *corresponds to a feasible flight plan satisfying the original continuity and turn-time conditions.*

Proof. Fix an aircraft j and list its assigned flights in increasing departure-time order as $i_1, \ldots, i_m$. For each consecutive pair, the intervals I_{i_k} and $I_{i_{k+1}}$ cannot intersect: otherwise $\{i_k, i_{k+1}\} \in \mathcal{O}$ and the C5 inequality would prohibit both flights from being assigned to j. Since the intervals are half-open and ordered by departure time, this gives

$$t^{\uparrow}_{i_{k+1}} \geq t^{\downarrow}_{i_k} + \Delta t, \qquad k = 1, \ldots, m-1.$$

Thus the aircraft has the required turn time between every consecutive pair of its assigned flights. Constraints (4)–(5) ensure that each assigned flight departs from an airport at which the aircraft is available: at a non-home airport, the cumulative balance requires a prior arrival with the required turn time; at the initial airport, the initial aircraft supplies the first available unit and the balance accounts for later returns and departures. Therefore the ordered flights form a continuous, turn-time-feasible route for aircraft j. Applying this argument to every aircraft proves the claim. □

4. The Maintenance-Extended Model and Cumulative-Flight-Time Correction

4.1. Maintenance problem definition

Following Khaled et al. (2018), we consider type-A checks triggered after at most $T^A_{\max}$ cumulative flight hours. Checks take place at night after the last flight on a given day in an airport with maintenance capacity.

Additional notation is collected in Table 2.

Table 2: Additional notation for the maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \ldots, n_d\}$	Planning days
d_i	Departure day of flight i
$\bar{d_i}$	Arrival day of flight i
$\mathcal{M} \subseteq \mathcal{A}$	Maintenance-capable airports
$\text{mcap}_{m,d}$	Maintenance capacity of airport m on day d
mc_{ijd}	Cost of night maintenance at arrival airport of flight i, aircraft j, day d
$F_d^{\downarrow}(m)$	Flights arriving at airport m on day d
$F(d, i)$	$\{i' : d_{i'} = d,\ t_{i'}^{\uparrow} > t_i^{\downarrow}\}$
$T_{\max}^{A}$	Max cumulative flight minutes before type-A check
$d_{\max}$	Max calendar-day interval between checks
$M_{dd'}$	Big-M for day pair (d, d')
$F_{d,d'}$	Flights with departure day in $(d, d']$

4.2. Additional decision variables

C6, C7 — Boolean variables for maintenance..

$$z_{ijd} = \begin{cases} 1 & \text{night maintenance at arrival airport of } i,\ \text{aircraft } j,\ \text{day } d, \\ 0 & \text{else;} \end{cases}$$

$$y_{jd} = \begin{cases} 1 & \text{maintenance of aircraft } j \text{ on day } d, \\ 0 & \text{else,} \end{cases}$$

The variable z_{ijd} is instantiated only when flight i arrives at a maintenance-capable airport on day d. Thus its admissible domain is $i \in \bigcup_{m \in \mathcal{M}} F_d^{\downarrow}(m)$. We assume in this type-A model that a night check fits within the overnight

ground period.

4.3. Extended model

The objective becomes:

$$\min \sum_{i\in\mathcal{F}}\sum_{j\in\mathcal{P}} c_{ij}x_{ij} + \sum_{d\in\mathcal{D}}\sum_{m\in\mathcal{M}}\sum_{i\in F_d^{\downarrow}(m)}\sum_{j\in\mathcal{P}} mc_{ijd}\, z_{ijd}. \tag{11}$$

Constraints (1)–(8) are retained. Additional constraints:

C8 — Maintenance blocks same-day subsequent flights..

$$z_{ijd} + x_{i'j} \le 1, \quad \forall d \in \mathcal{D},\ m \in \mathcal{M},\ i \in F_d^{\downarrow}(m),\ i' \in F(d,i),\ j \in \mathcal{P}. \tag{12}$$

C9 — Maintenance requires assignment..

$$x_{ij} \ge z_{ijd}, \quad \forall d \in \mathcal{D},\ m \in \mathcal{M},\ i \in F_d^{\downarrow}(m),\ j \in \mathcal{P}. \tag{13}$$

C10 — Capacity..

$$\sum_{i\in F_d^{\downarrow}(m)}\sum_{j\in\mathcal{P}} z_{ijd} \le \mathrm{mcap}_{m,d}, \qquad \forall d \in \mathcal{D},\ m \in \mathcal{M}. \tag{14}$$

C11 — Aggregation ($z \to y$)..

$$\sum_{m\in\mathcal{M}}\sum_{i\in F_d^{\downarrow}(m)} z_{ijd} = y_{jd}, \qquad \forall d \in \mathcal{D},\ j \in \mathcal{P}. \tag{15}$$

C12 — Maximum-spacing constraint..

$$\sum_{r\in[d,d']} y_{jr} \ge 1, \qquad \forall d, d' \in \mathcal{D} \text{ s.t. } d' - d = d_{\max} - 1,\ \forall j \in \mathcal{P}. \tag{16}$$

For the type-A-only model studied in this paper, C12 is not an additional calendar-spacing requirement: type-A maintenance is governed by the exact

flight-hour state introduced in the cumulative-state formulation below. The displayed row is retained as the generic day-indexed maintenance notation and becomes active only for a calendar-time maintenance type, which is outside the scope of this paper.

The original paper's cumulative-flight-time constraint is C13:

C13 — Paper-based cumulative-flight-time constraint..

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^{A}_{\max} + M_{dd'}\Big(2 - y_{jd} - y_{jd'}\Big) + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \qquad (17)$$

for all $j \in \mathcal{P}$, all pairs (d, d') with $2 \le d' - d \le d_{\max} - 1$, $d < n_d$, where $\tilde{t}_i$ is the flight duration of leg i.

4.4. Impact of corrected C1–C5 feasibility on C8–C13

The correction introduced in Section 3 (non-overlap constraint (8)) changes the admissible assignment set for x, and therefore the interpretation of all maintenance constraints layered on top of x.

C8.. No algebraic change is required in (12); however, with (8) active, the blocking relation now operates on physically realizable trajectories only. This removes spurious cases where maintenance placement was evaluated on infeasible simultaneous departures.

C9–C12.. Constraints (13)–(16) remain structurally unchanged. Their feasible region is tightened indirectly because z and y are linked to a strictly smaller, physically valid x-space.

C13 and cumulative-hour rows.. Hour-accumulation constraints are evaluated over a physically executable x-space once (8) is imposed. This correction is necessary, but it does not by itself establish that a particular cumulative-hour algebraic form correctly propagates the maintenance counter; that property is analysed below.

In summary, the C1–C5 correction primarily changes the domain over which the maintenance constraints operate. It is independent of the separate algebraic correction required for cumulative-hour propagation.

4.5. Issues of day-indexed maintenance constraints C8–C13

Usage of day-indexed maintenance variables y_{jd} and z_{ijd} is operationally intuitive, gives clear modeling semantics, but it has the following drawbacks:

- The day-indexed formulation requires a large number of big-M constants $M_{dd'}$ to cover all day pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$. This is a combinatorial number of rows, and the big-M values are necessarily loose because they must cover all possible flight-hour accumulations over the horizon.

- The day-indexed formulation does not propagate the flight-hour state exactly. The big-M relaxation allows the flight-hour counter to be reset prematurely, leading to potential violations of the intended maintenance intervals. For example, if a maintenance night is scheduled on day d and the next maintenance night is scheduled on day d', the big-M constraints only enforce that the cumulative flight hours between d and d' do not exceed $T^A_{\max}$, but they do not prevent excessive flight hours before day d.

- The day-indexed formulation forces a specific maintenance schedule on each day for a given time zone, thus allowing to have two different maintenance checks within 24 hrs, but not in two different days within the same 24-hr period.

- The day-indexed formulation does not allow for a natural integration with the ordered flight formulation, which is more efficient and compact. The ordered flight formulation allows for a more direct representation of the maintenance events in relation to the flight sequence, leading to a more efficient model.

- The original cumulative-flight-time row (paper equation) in the day-indexed formulation is flawed, as it does not correctly enforce the flight-hour limit between two consecutive maintenance nights. This flaw can lead to infeasible solutions that violate the intended maintenance intervals.

4.6. Legacy cumulative-flight-time row: original formulation and flaw

The original paper's cumulative-flight-time constraint (17) is intended to enforce that the cumulative flight hours between two consecutive maintenance nights do not exceed $T^A_{\max}$. However, as shown in Lemma 1, this constraint can fail to enforce the intended maintenance intervals in certain cases, particularly when there is no maintenance scheduled at the start of the interval.

Lemma 1 (Flaw in (17))**.** *If $y_{jd} = 0$ and $y_{jd'} = 1$, with no intermediate check, the RHS of* (17) *becomes $T^A_{\max} + M_{dd'}$. Because $M_{dd'}$ is chosen as*

a large upper bound on cumulative flight time, the row is then vacuous and imposes no limit on flying time before the first check of the period. The same issue occurs symmetrically when $y_{jd} = 1$ and $y_{jd'} = 0$.

Proof. Direct substitution of $y_{jd} = 0$, $y_{jd'} = 1$, $\sum_r y_{jr} = 0$ into (17) gives the stated RHS. Since $M_{dd'}$ is chosen as the maximum possible cumulative flight time over the horizon (a large positive constant), the resulting constraint is trivially satisfied regardless of the actual flight hours flown. □

4.7. Legacy cumulative-flight-time row: endpoint-split strengthening and limitation

An endpoint-split strengthening considered for replacing (17) consists of two constraints, each relaxed by one endpoint:

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{18}$$

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd'} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{19}$$

for all $j \in \mathcal{P}$, $(d, d') \in \mathcal{D}^2$ with $2 \le d' - d \le d_{\max} - 1$, $d < n_d$.

Lemma 2 (The endpoint split is not sufficient)**.** *With $F_{d,d'}$ defined as the flights departing in $(d, d']$, constraints* (18)–(19) *do not necessarily bound all flying time between two consecutive maintenance nights.*

Proof. Let $T^A_{\max} = 10$, $M_{dd'} = 20$, $d_{\max} \ge 4$, and consider days $1, \ldots, 4$ with $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. Assign 10 units of flying time on day 2 and 10 units on day 4. For $(d, d') = (1, 3)$, the accumulated time is 10 and the second split inequality is active. For $(2, 4)$, the accumulated time is 10

and the first split inequality is active. For $(1, 4)$, the accumulated time is 20, but both inequalities have right-hand side $T^A_{\max} + M_{14} = 30$ because both endpoints contain checks. Thus every split inequality is satisfied although 20 units are flown between the consecutive checks, exceeding the limit 10. □

The original and split formulations are therefore not ordered by strength: the original row is active when both endpoints contain checks, whereas the split rows are active when at least one corresponding endpoint indicator is zero. Neither pattern alone gives an exact cumulative counter.

4.8. Exact cumulative-state formulation for cumulative flight hours

Define the flying time assigned to aircraft j on day d by

C13a Daily flight time.

$$v_{jd} = \sum_{i \in \mathcal{F}:\, d_i = d} \tilde{t}_i x_{ij}. \tag{20}$$

Let $h_{jd} \geq 0$ be the accumulated type-A flying time immediately after the flights of day d and before maintenance that night. Set $h_{j0} = \Phi^A_j$ and $y_{j0} = 0$. For consecutive days, impose

C13b-C13f Accumulated flight time constraints.

$$h_{jd} \geq h_{j,d-1} + v_{jd} - M^H y_{j,d-1}, \tag{21}$$

$$h_{jd} \leq h_{j,d-1} + v_{jd} + M^H y_{j,d-1}, \tag{22}$$

$$h_{jd} \geq v_{jd}, \tag{23}$$

$$h_{jd} \leq v_{jd} + M^H (1 - y_{j,d-1}), \tag{24}$$

$$0 \leq h_{jd} \leq T^A_{\max}, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}, \tag{25}$$

where $M^H \geq T^A_{\max}$ is valid because $h_{j,d-1} \leq T^A_{\max}$. If $y_{j,d-1} = 0$, the first two inequalities give $h_{jd} = h_{j,d-1} + v_{jd}$. If $y_{j,d-1} = 1$, the reset inequalities give $h_{jd} = v_{jd}$. The upper bound then enforces the flight-hour threshold on every day, including the flights operated immediately before a night check.

The state h_{jd} is indexed at day boundaries, whereas the ordered Event-based state in Section 5 is indexed after individual flights and resets at a selected event. These are distinct time resolutions; no general projection-equivalence theorem between the formulations is claimed here. Section 6.3 compares them under the stated A-check timing policies and independently audits optimizer-selected schedules.

4.9. Equivalent implementation strengthening

The exact-state formulation admits a smaller, tighter implementation without changing its integer feasible region. The strengthened implementation is type-A-only, matching the scope of this paper. Its state variable receives the threshold directly as a variable bound, $0 \leq h_{jd} \leq T^A_{\max}$. The explicit copy of this upper bound in (25) is then unnecessary in the generated matrix.

Define the aircraft-day upper bound

$$\bar{v}_{jd} = \sum_{i \in \mathcal{F}:\, d_i = d,\, (i,j) \in \mathcal{X}} \tilde{t}_i \tag{26}$$

and the reachable state bounds

$$U_{j0} = \min\{T^A_{\max}, \Phi^A_j\}, \qquad U_{jd} = \min\{T^A_{\max}, U_{j,d-1} + \bar{v}_{jd}\}. \tag{27}$$

Because $h_{jd} \leq U_{jd}$ is valid by induction, the reset linearization for day d may use $U_{j,d-1}$ instead of the horizon-wide $T^A_{\max}$. This is particularly tighter near the beginning of the planning horizon.

Finally, the strengthened implementation generates maximal interval-clique rows for flight overlap. The implementation is `LegacyCorrectedStrengthenedMILPSchedule` the original `LegacyCorrectedMILPScheduler` remains available for the earlier four-method baseline; the current five-method scale-up uses the strengthened class.

4.10. Pre-horizon accumulated hours

The original paper does not account for flying time accumulated before the planning horizon. In the cumulative-state formulation this history is included exactly through the initial condition

$$h_{j0} = \Phi_j^A, \qquad 0 \le \Phi_j^A \le T_{\max}^A, \quad \forall j \in \mathcal{P}. \tag{28}$$

The condition is required for every aircraft, including $\Phi_j^A = 0$; no separate opening-window big-M inequality is needed.

4.11. Cumulative-prefix representation of strict interval rows

The strict interval form of C13/C13b also admits a cumulative-prefix representation. This is an alternative encoding of the interval rows, not the reset state h_{jd} above. Let q_{jd} be the flight minutes assigned to aircraft j on day d, and define $Q_{j0} = 0$ and

$$Q_{jd} = Q_{j,d-1} + q_{jd}, \qquad 0 \le Q_{jd} \le U_j^{\text{fly}}, \tag{29}$$

where U_j^{fly} is any valid upper bound on the total assignable flight minutes for aircraft j. For every day pair $d < d'$, telescoping gives

$$\sum_{i:\, d < \text{depday}(i) \le d'} \tilde{t}_i x_{ij} = Q_{jd'} - Q_{jd}, \qquad \sum_{i:\, \text{depday}(i) \le d'} \tilde{t}_i x_{ij} = Q_{jd'}. \tag{30}$$

Consequently, the strict interval rows can be written as

$$Q_{jd'} - Q_{jd} \le T^A_{\max} + M_{dd'} \sum_{r=d+1}^{d'-1} \mu_{jr}, \tag{31}$$

$$Q_{jd'} \le T^A_{\max} - \Phi^A_j + M^0 \sum_{r=1}^{d'-1} \mu_{jr}, \tag{32}$$

where μ_{jr} is the A-check reset indicator used by the strict interval formulation. Since the prefix flow equations uniquely determine Q from the assignment variables and the rows above are exactly the telescoped interval expressions, this substitution preserves the projection onto the common variables and the bound-preserving LP objective. It does not assert LP equivalence with the stronger day-indexed reset-state formulation h_{jd}.

4.12. Integrality constraints

$$x_{ij} \in \{0,1\}, \quad z_{ijd} \in \{0,1\}, \quad y_{jd} \in \{0,1\}, \qquad \begin{array}{l} (i,j) \in \mathcal{F} \times \mathcal{P}, \\ d \in \mathcal{D}, \quad i \in \bigcup\limits_{m \in \mathcal{M}} F^{\downarrow}_d(m) \text{ for } z_{ijd}. \end{array} \tag{33}$$

The variables h_{jd} are continuous and satisfy $0 \le h_{jd} \le T^A_{\max}$ as imposed in (25).

As we see, the fixes to the original paper's maintenance model are twofold: (i) the C0–C5 correction, which ensures that the flight assignment is physically feasible, and (ii) the cumulative-flight-time row correction. Also, the cumulative-flight-time correction is implemented in two ways: (i) the endpoint-split strengthening, which is a partial fix, and (ii) the exact cumulative-state formulation, which is a complete fix. The exact cumulative-state formulation

correctly propagates the flight-hour state and enforces the maintenance intervals as intended. The complexity of the corrected model is higher than the original model, because we need to track the cumulative flight hours explicitly that require more $O(n_f D)$ variables, but it is necessary to ensure that the solutions are physically feasible and respect the maintenance requirements. day-indexed model is higher than the original model, because the cumulative flight-hour state h_{jd} adds $O(n_p n_d)$ continuous variables,

The additional complexity is justified by the need to accurately model the maintenance requirements and ensure that the resulting flight schedules are feasible in practice.

4.13. Rigorous feasibility theorem for C1–C13f

We now state the corrected sufficiency result for the type-A maintenance model. Consider the constraint family (1)–(8), (12)–(15), (21)–(25), and the stated variable domains. Constraint C12 is inactive for the type-A-only model: hour spacing is enforced by the cumulative-state rows rather than by calendar-day covering.

Theorem 1 (Physical feasibility under corrected C1–C13f)**.** *Any solution satisfying the above constraints, with x, z, and y binary and h continuous, induces for each aircraft j a physically executable trajectory over the planning horizon: flights assigned to j are temporally non-overlapping, satisfy airport continuity and turn time, and admit a day-by-day maintenance schedule satisfying capacity and hour-spacing rules.*

Proof. Fix an aircraft j and let $S_j = \{i \in \mathcal{F} : x_{ij} = 1\}$ ordered by nondecreasing departure time.

(i) Flight assignment and continuity. By (3), every flight is assigned to exactly one aircraft. By (4)–(5), each departure assigned to j has sufficient prior balance at its departure airport, with the initial unit at a_j^0. Therefore airport continuity holds along S_j.

(ii) Temporal executability. For any overlapping pair $\{i_1, i_2\} \in \mathcal{O}$, (8) enforces $x_{i_1 j} + x_{i_2 j} \leq 1$. Hence no two flights assigned to j overlap once turn time is included. Combined with (i), this yields a realizable single-aircraft flight timeline.

(iii) Maintenance event consistency. By (13), maintenance trigger variables can only be activated on assigned flights. By the aggregation equality (15), each day's maintenance trigger is represented by the single aggregate indicator y_{jd}; by the binary day indicator in (15), the maintenance triggers on day d are represented by the corresponding aggregate maintenance state.

(iv) Maintenance capacity and blocking. By (14), airport-day capacities are respected. By (12), if maintenance is triggered after flight i on day d, then any subsequent same-day flight is forbidden for the same aircraft. Under the stated overnight-duration assumption, the maintenance action is operationally executable.

(v) Hour/day maintenance admissibility. By (21)–(25), the cumulative flight-time state increases by the flying time assigned each day, resets after a maintenance night, and never exceeds $T_{\max}^A$.

Steps (i)–(v) show that every aircraft has a consistent, non-overlapping flight sequence with feasible maintenance placement and resource usage. Therefore the global solution is physically feasible. □

5. Ordered Event-Based Reformulation for a Flight-hour Maintenance

In this paper, we present an ordered event-based model for flight-hour maintenance. It removes day-indexed maintenance decision states and does not introduce the standalone successor-arc event formulation. Flights are processed in nondecreasing departure-time order, and maintenance is represented by a binary event attached to the flight that triggers it. Thus the design-note variables x_{ij} and z_{ij} are represented below through an ordered position index r that maps to a flight index i_r.

The formulations in this section are for type-A checks only. The optional flexible-start variant E14$_{mathrmflex}$–E21$_{mathrmflex}$ also models only A-check timing; it does not include the B/C/D maintenance hierarchy. Its computational implementation uses a one-minute time grid and a 1440-minute maximum deferral window.

5.1. Ordered event representation

For flight i, let o_i and a_i be its departure and arrival airports, p_i its flying time, and $t_i^{\uparrow}$ and $t_i^{\downarrow}$ its departure and arrival timestamps (the same macros as in Section 3). For aircraft j, let a_j^0 be its initial airport.

Define the ordered-position index set

$$\mathcal{R} = \{1, \ldots, n_f\},$$

and let $(i_1, \ldots, i_{n_f})$ be flights sorted by $(t_i^{\uparrow}, i)$. Position $r \in \mathcal{R}$ refers to flight i_r. This is the only role of r: it is an order index over flights. Define

$$\mathcal{R}^M = \{r \in \mathcal{R} : i_r \in \mathcal{F}^M\},$$

and let $\mathcal{Z}$ be the set of admissible event pairs (r, j) for which flight i_r can trigger the single type-A check on aircraft j. Use the following A-only variables:

E1-E3. Ordered event variables..

- $x_{rj} \in \{0, 1\}$: aircraft j operates flight i_r;
- $z_{rj} \in \{0, 1\}$: a type-A maintenance event is performed after flight i_r on aircraft j;
- $h_{rj} \geq 0$: cumulative type-A flight time of aircraft j immediately after processing position r.

Because $r \leftrightarrow i_r$ is one-to-one, the relation to the original notation is direct: $x_{rj} = x_{i_r j}$ and $z_{rj} = z_{i_r jd}$ for the corresponding arrival day d; the check type is always A in this section.

A maintenance event after flight i_r is assignment-linked and starts at its arrival time. The corrected airport-continuity, turn-time, and overlap constraints remain active on the assignment variables. In particular, an assignment is allowed only when it can be connected to the aircraft's preceding assigned flight and when the required maintenance duration leaves enough time before the next assigned departure.

The objective is

$$\min \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{P}} c_{rj} x_{rj} + \sum_{r \in \mathcal{R}^M} \sum_{j \in \mathcal{P}} mc_{rjA}\, z_{rj}. \tag{34}$$

The event formulation numbers its route constraints independently as follows: E4 is flight coverage, E5 is non-home continuity, E6 is home-airport

continuity and turn-time balance, and E7 is pairwise non-overlap. These are implemented by `_add_c1_coverage`, `_add_c23_turn`, and `_add_overlap`. E1–E3 are the binary/continuous variable domains defined above.

The event constraints separate maintenance blocking (E14), event assignment (E8), and maintenance capacity (E15).

E8. Event assignment.. Events satisfy

$$z_{rj} \leq x_{rj}, \qquad \forall (r, j) \in \mathcal{Z}, \tag{35}$$

The integer event-count link in (60) is an auxiliary C11 aggregation, not a separate E family. Timestamp-based incompatibility rows are E14; they prevent an event from being followed by a departure before the maintenance duration and turn time have elapsed.

5.2. Prefix-state propagation

Let $T_F := T^A_{\max}$ be the maximum flying time between type-A maintenance events and let Φ_j be the pre-horizon accumulated flying time of aircraft j. At every ordered position, an unassigned flight carries the state forward, an assigned flight adds its duration, and an event resets the state after its triggering flight. Using valid position-specific big-M values M_{rj}, the transition is represented by

E9–E13. Prefix-state propagation..

$$h_{rj} \geq h_{r-1,j} - M_{rj} x_{rj}, \tag{36}$$

$$h_{rj} \leq h_{r-1,j} + M_{rj} x_{rj}, \tag{37}$$

$$h_{rj} \geq h_{r-1,j} + p_r - M_{rj}(1 - x_{rj}) - M_{rj} z_{rj}, \tag{38}$$

$$h_{rj} \leq h_{r-1,j} + p_r + M_{rj}(1 - x_{rj}) + M_{rj} z_{rj}, \tag{39}$$

$$h_{rj} \leq T_F(1 - z_{rj}), \tag{40}$$

$$h_{r-1,j} + p_r \leq T_F + M_{rj}^{\text{pre}}(1 - x_{rj}) \tag{41}$$

The first two rows (E9–E10) carry the state at an unassigned position. The next two rows (E11–E12) impose accumulation for an assigned flight unless an event is selected. The first E13 row resets the post-event state to zero when $z_{rj} = 1$. The second E13 row is the pre-event threshold guard: the flight cannot arrive over the flight-hour limit and then use the event to reset the state. Here M_{rj}^{pre} is sufficiently large to deactivate the guard when $x_{rj} = 0$. The initial state is $h_{0j} = \Phi_j$.

5.3. Maintenance blocking and capacity constraints

Let's assume that every maintenance event starts immediately after its triggering flight. Let δ_A be the duration of the type-A check and let Δt be the minimum post-maintenance turn time.

E14. Maintenance blocking (C8).. For an event (r, j), define the incompatible following-flight set

$$F_r^{\text{dep}} = \left\{ \ell \in \mathcal{F} : \; a_\ell^{\uparrow} = a_{i_r}^{\downarrow}, \; t_{i_r}^{\downarrow} \leq t_\ell^{\uparrow} < t_{i_r}^{\downarrow} + \delta_A + \Delta t \right\}. \tag{42}$$

If $z_{rj} = 1$, every selected incompatible flight is blocked:

$$x_{\ell j} + z_{rj} \leq 1, \qquad \forall \ell \in F_r^{\text{dep}},\ j \in \mathcal{P}. \tag{43}$$

Only feasible (ℓ, j) assignment arcs are generated. Restricting ℓ to departures from the station $a_{i_r}^{\downarrow}$ where the check takes place loses nothing: by E5–E6 the aircraft cannot leave from anywhere else without first operating an intervening flight, which the same family already blocks.

For an admissible assignment arc, the same blocking condition can be written with an arc-specific big-M value:

$$\begin{aligned} t_\ell^{\uparrow} + M_{r\ell}(2 - x_{\ell j} - z_{rj}) \geq t_{i_r}^{\downarrow} + \delta_A + \Delta t, \\ \forall r \in \mathcal{R}^M,\ \ell \in \mathcal{F},\ j \in \mathcal{P}, \\ (\ell, j) \text{ admissible},\ a_\ell^{\uparrow} = a_{i_r}^{\downarrow},\ t_\ell^{\uparrow} \geq t_{i_r}^{\downarrow}. \end{aligned} \tag{44}$$

E15. Maintenance capacity (C10).. For each possible event start $\theta_r := t_{i_r}^{\downarrow}$, define the active event set

$$\mathcal{E}(a, \theta) = \left\{(q, j) \in \mathcal{Z} : a_{i_q}^{\downarrow} = a, \quad \theta_q \leq \theta < \theta_q + \delta_A\right\}, \qquad \theta_q := t_{i_q}^{\downarrow}. \tag{45}$$

At every possible event start, active maintenance occupancy satisfies

$$\sum_{(q,j) \in \mathcal{E}(a, \theta_r)} z_{qj} \leq \text{mcap}_a, \qquad \forall a \in \mathcal{M},\ r \in \mathcal{R}^M. \tag{46}$$

Each event occupies the half-open interval $[\theta_q, \theta_q + \delta_A)$. Since all A-check windows have equal length, checking occupancy at every event start enforces capacity at every time.

The constraints 43 and 46 enforce maintenance blocking and capacity limitations, however they reduce the solution space by imposing additional restrictions on the feasible assignments of flights and maintenance events.

Potentially, if the plain A instead of going directly to maintenance, waits and during this waiting period another plane B arrives and occupies the maintenance slot, the constraints will prevent the initial plane from being assigned to maintenance at that time, even if a delayed maintenance could be operationally feasible.

To fix the issue of overly restrictive maintenance assignment due to simultaneous occupancy constraints, we introduce another variable representing the flexible completion time of maintenance events. This variable allows the model to adjust the timing of maintenance dynamically, ensuring that aircraft can be assigned to maintenance even if the initially preferred time slot is occupied. Let W be the maximum allowed waiting time between the triggering flight's arrival and the start of its A-check. We use a time grid of resolution τ (one minute for the instances considered here); all timestamps, δ_A, W, and maintenance offsets are multiples of τ. For each admissible event (r, j), let s_{rj} be the offset from the arrival of i_r to the completion of the check. Its maintenance interval is

$$\left[t^{\downarrow}_{i_r} + s_{rj} - \delta_A,\ t^{\downarrow}_{i_r} + s_{rj}\right), \tag{47}$$

so $s_{rj} = \delta_A$ gives immediate start and $s_{rj} - \delta_A$ is the deferral. Define $\mathcal{B}^W$ as the set of triples (r, ℓ, j) for which $(r, j) \in \mathcal{Z}$, (ℓ, j) is an admissible assignment arc, $a^{\uparrow}_{\ell} = a^{\downarrow}_{i_r}$, and $t^{\downarrow}_{i_r} \le t^{\uparrow}_{\ell} < t^{\downarrow}_{i_r} + \delta_A + W + \Delta t$.

E14$_{\text{flex}}$. *Aircraft blocking..* If event (r, j) is selected, aircraft j cannot depart on a following flight until the check and post-maintenance turn time are complete:

$$t^{\uparrow}_{\ell} + M_{r\ell}(2 - x_{\ell j} - z_{rj}) \ge t^{\downarrow}_{i_r} + s_{rj} + \Delta t, \qquad \forall (r, \ell, j) \in \mathcal{B}^W. \tag{48}$$

Here $M_{r\ell}$ is an event–flight-specific valid relaxation value; for example, it may be chosen no smaller than $\max\{0, t^{\downarrow}_{i_r} + \delta_A + W + \Delta t - t^{\uparrow}_{\ell}\}$.

E15$_{\text{flex}}$. *Deferral bounds..* The event cannot finish before the A-check could be completed, and its start cannot be delayed by more than W:

$$\delta_A z_{rj} \le s_{rj} \le (\delta_A + W) z_{rj}, \qquad s_{rj} \in \tau\mathbb{Z}_+, \quad \forall (r,j) \in \mathcal{Z}. \tag{49}$$

Thus $s_{rj} = 0$ when the event is not selected.

E16$_{\text{flex}}$. *Realized event time..* By E4 and E8, at most one aircraft can be assigned an event after any one flight. Define the selected-event indicator ζ_r, the realized offset S_r, and its completion time T_r by

$$\zeta_r = \sum_{j:(r,j)\in\mathcal{Z}} z_{rj}, \qquad S_r = \sum_{j:(r,j)\in\mathcal{Z}} s_{rj}, \qquad T_r = t^{\downarrow}_{i_r} + S_r, \qquad r \in \mathcal{R}^M. \tag{50}$$

The explicit uniqueness row E21$_{\text{flex}}$ below records the at-most-one property used by these aggregates.

Let $\mathcal{W}$ contain the ordered pairs of event-trigger flights at the same maintenance airport, including every diagonal pair (r, r). A pair may be omitted only if bounds on its possible event times prove the two maintenance intervals cannot overlap. For each distinct $(r, q) \in \mathcal{W}$, let $D_{rq} = T_q - T_r$. Binaries $u_{rq}, v_{rq} \in \{0, 1\}$ encode the two inclusive grid comparisons that characterize whether event q is active at the start of event r. Let $u_{rq} = 1$ exactly when $D_{rq} \le 0$ and $v_{rq} = 1$ exactly when $D_{rq} \ge -\delta_A + \tau$. Since $T_r \in [t^{\downarrow}_{i_r}, t^{\downarrow}_{i_r} + \delta_A + W]$, one valid pair-specific choice is $M_{rq} \ge \max\{|L_{rq}|, |U_{rq}|\} + \delta_A + \tau$, where $L_{rq} = t^{\downarrow}_{i_q} - t^{\downarrow}_{i_r} - (\delta_A + W)$ and $U_{rq} = t^{\downarrow}_{i_q} - t^{\downarrow}_{i_r} + \delta_A + W$. These comparisons

are encoded by

$$D_{rq} \le M_{rq}(1-u_{rq}), \qquad D_{rq} \ge \tau - M_{rq}u_{rq}, \tag{51}$$

$$D_{rq} \ge -\delta_A + \tau - M_{rq}(1-v_{rq}), \qquad D_{rq} \le -\delta_A + M_{rq}v_{rq}, \tag{52}$$

for distinct r, q. The grid assumption makes the complements exact: if $D_{rq} > 0$, then $D_{rq} \ge \tau$; if $D_{rq} \not\ge -\delta_A + \tau$, then $D_{rq} \le -\delta_A$.

E19$_{\text{flex}}$. *Active-at-start indicator..* For each ordered pair at the same airport, let o_{rq} indicate that both events are selected and event q is active when event r begins, with $o_{rq} \in \{0,1\}$. For $r \ne q$, impose

$$\begin{aligned} &o_{rq} \le \zeta_r, && o_{rq} \le \zeta_q, \\ &o_{rq} \le u_{rq}, && o_{rq} \le v_{rq}, \\ &o_{rq} \ge \zeta_r + \zeta_q + u_{rq} + v_{rq} - 3. \end{aligned} \tag{53}$$

For diagonal pairs set $o_{rr} = \zeta_r$. Consequently, $o_{rq} = 1$ exactly when $\zeta_r = \zeta_q = 1$ and $T_q \in [T_r - \delta_A + \tau, T_r]$, which is precisely the half-open interval overlap test at the start of event r on the τ-grid.

E20$_{\text{flex}}$. *Station capacity..* All A-check intervals have equal duration, so maximum station occupancy is attained at the start of a selected event. At every such start, impose

$$\sum_{q:(r,q)\in\mathcal{W}} o_{rq} \le \text{mcap}_{a^{\downarrow}_{i_r}}\ \zeta_r, \qquad r \in \mathcal{R}^M, \tag{54}$$

The self term $o_{rr} = \zeta_r$ counts the event whose start is being checked.

E21$_{\text{flex}}$. *Event uniqueness..* At most one aircraft can receive an event after a given triggering flight:

$$\sum_{j:(r,j)\in\mathcal{Z}} z_{rj} = \zeta_r \leq 1, \qquad r \in \mathcal{R}^M. \tag{55}$$

This row is implied by E4 and E8, but is stated explicitly to make the event aggregates in (50) unambiguous.

The overlap rows are exact on the stated time grid; no binary is merely penalized toward indicating an overlap, and no capacity is inferred from a pairwise-neighbour count. Setting $s_{rj} = \delta_A z_{rj}$ recovers immediate maintenance timing, so E14$_{\text{flex}}$–E20$_{\text{flex}}$ reduce to E14–E15. The flexible formulation therefore contains the immediate-start feasible schedules and can only weakly improve their minimum assignment cost.

5.4. Implemented E1–E15 model and flexible-start specification

The following list gives the event-family components of the current immediate-start ordered A-only builder and separately identifies the optional flexible-start formulation. Legacy C labels are given only as cross-references.

E1–E3: variable domains. The variables x_{rj} and z_{rj} are binary and h_{rj} is nonnegative. These domains correspond to the assignment, event, and prefix-state variables.

E4: flight coverage (legacy C2). Each ordered flight is assigned exactly once:

$$\sum_{j\in\mathcal{P}} x_{rj} = 1, \qquad r \in \mathcal{R}. \tag{56}$$

E5–E6: aircraft continuity and turn time (legacy C3–C4). The corrected equipment-flow balance is applied to x_{rj}, including the initial aircraft position and the minimum turn time. The implementation is `e5_e6_continuity_turn`.

E7: pairwise non-overlap (legacy C5). For every temporally conflicting flight pair and aircraft, the builder adds

$$x_{ij} + x_{\ell j} \leq 1. \tag{57}$$

These rows are constructed by `e7_pairwise_overlap`; in the current ground-truth formulation they are the corrected C5 family.

The current ground-truth Sections 01–04 do not define a separate clique constraint number; any clique strengthening is an implementation strengthening of C5 rather than a new numbered requirement.

E8: event assignment (legacy C6–C7 and C9). The maintenance event variables z_{rj} are binary and are instantiated only for flights arriving at a maintenance-capable airport. There is no independent daily y variable in this formulation. Each admissible arrival can trigger at most one type-A event for an aircraft.

E14: maintenance blocking (legacy C8). If $z_{rj} = 1$, every selected flight whose departure falls before the event's completion plus turn time is blocked:

$$x_{\ell j} + z_{rj} \leq 1. \tag{58}$$

Only economically relevant timestamp conflicts are generated.

E15: maintenance capacity (legacy C10). At every possible event-start timestamp, the active arrival-started maintenance events satisfy

$$\sum_{(r,j)\in\mathcal{E}(a,\theta)} z_{rj} \le \mathrm{mcap}_a, \tag{59}$$

where $\mathcal{E}(a,\theta)$ contains events whose half-open intervals $[\theta_r, \theta_r+\delta_A)$ contain θ. This is implemented by the E15 component `e15_maintenance_capacity`; it is the maintenance-capacity constraint that was not explicitly displayed earlier.

E14$_{\text{flex}}$–E21$_{\text{flex}}$: deferrable maintenance timing (optional variant). The discrete-grid formulation in (48)–(55) replaces immediate-start E14–E15 by aircraft blocking, bounded completion offsets, exact overlap indicators, and capacity rows at selected event starts. It retains E1–E13 and includes immediate-start schedules as the special case $s_{rj} = \delta_A z_{rj}$. The overlap comparison binaries are required because the active-event set depends on the selected completion times. An implementation of this specification must enforce both overlap comparisons and their conjunction; an unconstrained or one-directional overlap indicator does not enforce capacity.

Auxiliary C11 event-count aggregation. The binary day indicator is intentionally not retained. Define the nonnegative integer event count n_{jd} by

$$n_{jd} = \sum_{r:\, \lfloor t^{\downarrow}_{ir}/1440 \rfloor = d} z_{rj}, \qquad n_{jd} \in \mathbb{Z}_+. \tag{60}$$

This preserves C11's aggregation information without imposing the removed one-event-per-day restriction.

Inactive legacy C12. Calendar spacing is inactive in the A-only model. It is implemented only by the C/D calendar extension, which is not enabled here.

E9–E13: cumulative flight-hour state (legacy C13–C13f). The paper's C13 row is replaced by the auxiliary prefix state h_{rj}, which is nonnegative, carries forward at unassigned positions, accumulates an assigned flight when $z_{rj} = 0$, resets to zero when $z_{rj} = 1$, and is bounded by $T^A_{\max}$. The daily flight-time definition is C13a and the state rows are C13b–C13f. These are implemented by `event_hour_state`; they replace the legacy day-pair C13 row rather than adding another numbered C-constraint.

The event formulation therefore has no additional business rule hidden under an unnumbered C-constraint. The auxiliary families are the integer C11 event counts and the linearization rows for the C13 hour state. The airport occupancy quantity $q_{a,j}$ proposed in the design note is not introduced as a variable: its role is represented directly by the C10 timestamp-capacity sums, which is smaller and equivalent for the A-only case.

5.5. Equivalent A-only implementation strengthening

The immediate-start E1–E15 formulation also admits an equivalent reduced implementation. The daily event count in (60) is not used by any E-constraint or by the objective, so it can be recovered after solution rather than represented by integer variables and equalities. With only one maintenance type, event-type exclusivity is implied by the binary domain of z_{rj} and is omitted.

The prefix state receives its valid upper bound directly as a variable bound. Consequently, the additional generated row that repeats this bound is redundant with E13 and is removed. Identical E15 active-event sets are emitted only once, and a capacity row is omitted whenever its number of binary terms does not exceed the capacity and is therefore satisfied for every assignment. Finally, maximal interval-clique rows replace the pairwise-plus-clique E7 implementation; every conflicting pair belongs to a maximal clique, so the integer feasible set is unchanged.

The baseline implementation remains `PaperEventBasedMILPScheduler`. The equivalent reduced implementation is `OptimizedPaperEventBasedMILPScheduler`; keeping both classes makes the effect of implementation choices independently reproducible. The exact flexible-start formulation is implemented by `FlexiblePaperEventBasedMILPScheduler`: its integer-grid offset, comparison binaries, active-at-start indicators, and station-capacity rows correspond to $\mathrm{E14}_{\mathrm{flex}}$–$\mathrm{E21}_{\mathrm{flex}}$. The earlier benchmark numbers for the previous flex linearisation are not results for this formulation. A separate 12-case flexible-prototype benchmark is recorded in extttresults/tables/flexible_event_benchmark_e14_e21.csv; it is not pooled with the six-instance, zero-day-legacy-deferral scale-up reported in Section 6.3.

Table 3 records the correspondence with the numbered day-indexed requirements. Some legacy rows disappear because their variables are eliminated; this is a formulation change, not an omitted feasibility condition.

Proposition 4 (Ordered prefix-state validity)**.** *Assume the corrected route constraints define a physically feasible integral assignment, every event is attached to an assigned flight at a maintenance-capable airport, and the sup-*

plied initial state is valid. Then the ordered prefix-state formulation satisfies the type-A flight-hour limit.

Proof. Induct on the ordered flight positions. An unassigned position carries the state unchanged. An assigned position adds its flying time unless an event is selected, in which case the reset rows set the state to zero after the current flight. The final transition row and the upper state bound prevent the threshold from being exceeded before a reset. The induction starts from Φ_j, so every assigned route satisfies the type-A limit. □

5.6. Why the ordered formulation is the better method

5.7. Theoretical complexity and model size

Let $n_f = |\mathcal{F}|$, $n_p = |\mathcal{P}|$, and let n_m denote the number of flights that arrive at a maintenance-capable airport. In the A-only ordered model, the assignment variables contribute at most $n_f n_p$ binary variables, while the maintenance-event variables contribute at most $n_m n_p$ binary variables. The aircraft-local prefix state contributes at most $n_f n_p$ continuous variables; with sparse aircraft eligibility, all three counts are smaller and are determined by the feasible (i, j) pairs. Thus the principal variable bound is

$$O(n_f n_p + n_m n_p),$$

with no additional day-indexed y_{jd} or occupancy-state $q_{a,j}$ family. The ordered implementation instead stores one state position per feasible flight–aircraft pair.

The main linear constraint families are $O(n_f)$ for coverage, $O(n_f n_p)$ for continuity and event assignment, and $O(n_f n_p)$ for the prefix-state equations.

C11 aggregation contributes $O(n_p D)$ equalities for the event counts, without restricting their values. Pairwise non-overlap and timestamp blocking depend on the conflict sets and are $O(|\mathcal{O}|n_p)$ and $O(|\mathcal{B}|)$, respectively, where $\mathcal{O}$ is the flight-overlap set and $\mathcal{B}$ is the generated set of event–following-flight blocking pairs. Timestamp-capacity rows are generated per airport breakpoint, so their number is proportional to the number of distinct event timestamps rather than to a full time grid. In the worst case, $|\mathcal{O}| = O(n_f^2)$ and the sparse conflict families can also be quadratic, but timetable sparsity usually makes their observed counts much smaller. The theoretical conclusion is therefore precise: the Event-based model removes the day dimension from the principal variable families and keeps prefix-state size linear in feasible flight–aircraft pairs; it does not guarantee fewer total constraints unless the induced conflict and blocking sets are also sparse.

The ordered formulation combines event-triggered maintenance with a global flight order and aircraft-local prefix states. It removes the day-pair accumulation ambiguity of the legacy cumulative-flight-time row, avoids successor-variable expansion, and keeps the physical route constraints needed for valid tail assignments. For type-A maintenance, C12's calendar-spacing role is inactive: the flight-hour limit is enforced by the exact prefix-state recursion. The computational study in Section 6 evaluates this formulation against the legacy model on matched horizon and fleet-size grids.

5.8. Complexity and model size of the flexible-start variant

The flexible variant keeps E1–E13 verbatim. It adds one grid-valued offset s_{rj} per admissible event, at most $n_m n_p$ such variables. The trigger-level quantities ζ_r, S_r, and T_r in (50) are linear expressions, not additional

variables. The principal variable count therefore remains $O(n_f n_p + n_m n_p)$ before the capacity-comparison binaries are included.

The E14_{flex} row set $\mathcal{B}^W$ spans the widened departure window $[t^{\downarrow}_{i_r}, t^{\downarrow}_{i_r} + \delta_A + W + \Delta t)$, so its size grows with W; the rows use big-M relaxation rather than the immediate-start set-packing form. For each ordered pair in $\mathcal{W}$, E17_{flex} and E18_{flex} use two binary comparisons, and E19_{flex} uses one overlap binary. E20_{flex} contributes one capacity row per possible trigger event, while E21_{flex} is implied by E4 and E8. Thus the added capacity model has $O(|\mathcal{W}|)$ binaries and rows. Without pair pruning, $|\mathcal{W}| = O(n_m^2)$; a pair may be pruned only when its completion-time bounds prove that its windows cannot overlap. The grid resolution is essential to exactness: it replaces strict half-open interval comparisons by finite, inclusive integer comparisons.

Fixing $s_{rj} = \delta_A z_{rj}$ recovers the immediate-start maintenance intervals. E14_{flex} then reduces to E14, and the capacity rows check the same occupancy at each selected event start as (46). Hence every immediate-start feasible schedule is feasible for the flexible formulation, and its optimal assignment cost can only be weakly lower. The aircraft remains grounded until the deferred check and post-maintenance turn time are complete; the additional freedom concerns maintenance timing, not flight operation while waiting.

6. Computational Study

6.1. Implementation and environment

All models are implemented in Python using Pyomo. The reported exact solves use unrestricted IBM ILOG CPLEX 12.6 for AMPL through Pyomo's `cplexamp` interface. The deterministic instance generator and all bench-

mark scripts are included in the companion repository. The generator independently validates each designated seeded rotation with its maintenance counter; MILP objectives and statuses are taken from the solver runs.

We also smoke-tested the Pyomo interface with Gurobi 11.0.3 on small feasible and infeasible models. On Windows, the `gurobi` shell interface runs through the Python interpreter bundled with Gurobi (discovered from `GUROBI_HOME`), rather than through `gurobi_cl.exe`. The infeasible case returns no incumbent and marks its problem bounds unavailable; the implementation preserves that termination status and omits schedule extraction. This is an interface check, not a benchmark comparison: all computational results and timing tables in this paper use CPLEX.

6.2. Cumulative-flight-time row verification

The original cumulative-flight-time row and the natural endpoint-split replacement were evaluated on the hand-built four-day counterexample `data/instances/c13_loopho` The instance fixes one aircraft, $T^A_{\max} = 600$ minutes, $M_{14} = 1200$ minutes, 1,200 minutes of flying in $(1, 4]$, and two check patterns: checks at both endpoints, and a check only at the end. The actual production row objects admit complementary violating patterns: the original row is relaxed when only one endpoint is checked, while the split replacement is relaxed when both endpoints carry checks. The exact cumulative-state and ordered event-state formulations reject both patterns. This directly confirms the algebraic analysis in Sections 4.6 and 4.7.

The flexible-start event variant is omitted from this table because it inherits E9–E13 verbatim from the optimized event model and therefore reproduces its verdicts on both patterns; the suite run in Section 6.3 confirms

this.

6.3. Five-method type-A comparison

The ordered implementation removes explicit successor variables and propagates an aircraft-local prefix state through flights sorted by departure time. A maintenance event is attached to its triggering flight, and the pre-check row prevents an event from repairing an already overdue arrival. The five methods evaluated in the matched A-only experiment are the original Khaled row, the endpoint split, the strengthened corrected legacy rows, the optimized ordered event-based formulation, and its flexible-start variant. Their builder identifiers are respectively `legacy_paper_c13`, `legacy_endpoint_split`, `legacy_corrected_strengthe` `event_based_optimized`, and `event_based_flex`. A result is described as objective-matched only when both compared methods are optimal. The legacy methods use a zero-day trigger-to-check deferral cap in the matched rerun; the optimized event model starts maintenance after the triggering flight, while the flexible event model permits a one-day (1440-minute) offset. The day-indexed and event-based timing rules remain different, so this setting improves comparability but does not make the formulations identical.

6.3.1. Longer horizons and fleet sizes

The five methods were run on six matched A-only instances generated in `data/five_method_longflight_e14_e21_compact/`. Fleet sizes are $|P| \in \{5, 10, 15\}$, observed horizons are $H \in \{30, 48, 49\}$, and the instances contain 74–240 flights with up to eight seeded round-trip rotations per aircraft. Each rotation is checked by the generator's independent maintenance replay; this validates the designated generated rotations, not every schedule returned by

a MILP. All five formulations use the same JSON instance within each cell. CPLEX 12.6 was run with one thread, presolve enabled, and a 180-second limit. Legacy methods were run with `max_hour_check_deferral_days=0`; event-flex retains its 1440-minute deferral window. Results, input hashes, and solver configuration are recorded in `results/tables/a_only_policy_matched_20261020.csv` and its metadata sidecar. This is an extended-family experiment, not a reproduction of Khaled et al.'s Table 6.

Table 5 reports solver status, incumbent objective, timing, model size, and independent replay status for all 30 performance runs. Both Event-based methods reached the time limit at P=15 for H=30 and H=49; their incumbents are not proven optima and were not independently replayed.

With zero-day legacy deferral, the strengthened legacy and both Event-based formulations have identical optimal objectives on the four cells where all three methods prove optimality. On the two P=15 cells, the Event-based methods return the same incumbent as strengthened legacy but hit the time limit, so that agreement is not an optimality certificate. The original-paper and endpoint-split models return lower objectives, but their independently replayed schedules exceed the A-hour threshold (663 and 183 violations across the six cases, respectively); those objectives are not feasible-quality comparisons. The optimized Event-based model has fewer variables than strengthened legacy on three of six cells; averaged over all six, its variable count is slightly higher. Neither Event-based formulation consistently reduces constraints or timing. These results are limited to this generated family and solver setup.

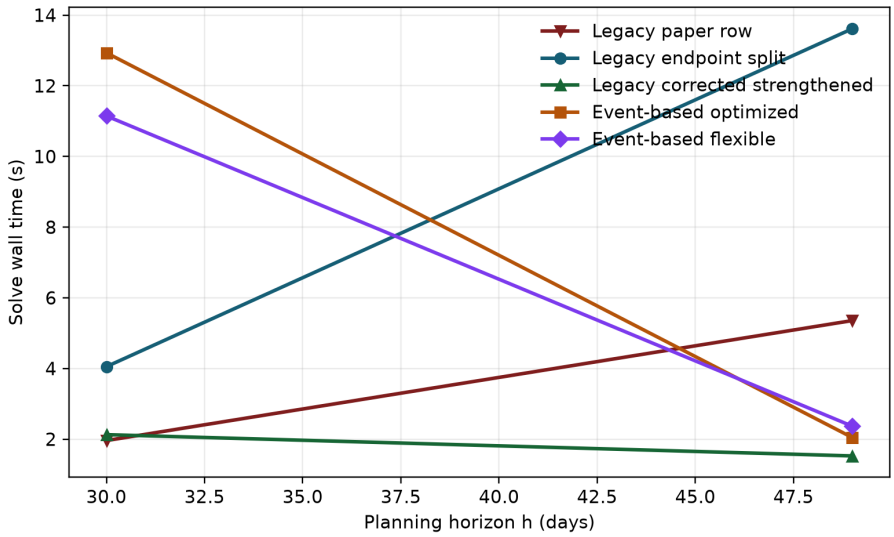


Figure 1: A-only solve wall time versus observed horizon for $|P| = 10$. Only paired cells where all five methods are optimal are plotted.

6.3.2. Fleet-size scaling at $H = 30$

The fleet-size figures use the complete matched set at observed horizon $H = 30$ for $|P| \in \{5, 10, 15\}$. Model-size figures include all successfully built models; timing figures include only cells where all five methods are optimal (P=5 and P=10).

6.4. Summary and limitations

All 30 solver runs returned incumbents and 26 terminated optimally. The four time-limited Event-based runs are the P=15 cases at H=30 and H=49; their incumbents are not optimality certificates and are excluded from paired timing averages. Independent replay reports zero violations for all six strengthened legacy schedules and the four optimal schedules from each Event-based method. The original-paper and endpoint-split schedules

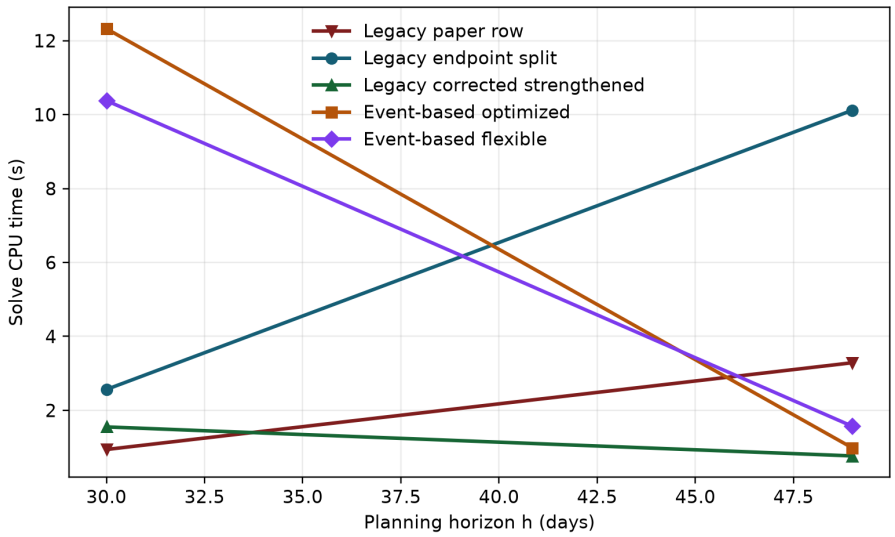


Figure 2: A-only solve CPU time versus observed horizon for $|P| = 10$. Only paired cells where all five methods are optimal are plotted.

fail replay in all six cells, consistent with the C13 counterexamples above. Time-limited Event-based schedules were not replayed.

The experiment uses one deterministic seed per (P, H) cell, one maintenance-capable hub, A checks only, and a single CPLEX configuration. It should be read as a reproducible scale-up test, not broad statistical evidence. Follow-up validation should cover multiple seeds, varied airport networks and station capacities, nonzero initial flight-hour counters, and independent schedule replay for each formulation. Solver comparisons should vary presolve and warm-start settings and report optimality gaps and memory alongside the metrics collected here.

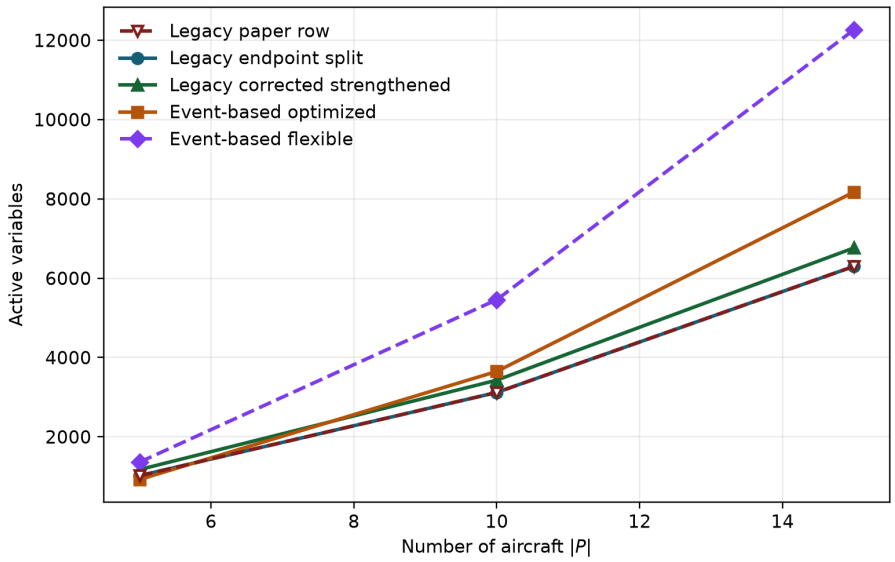


Figure 3: Active Pyomo variables versus fleet size at observed horizon $H = 30$.

6.4.1. Aggregate model size and optimal-run timing

Table 6 averages model counts over all six performance cases. Timing averages use the same four P=5/10 cells, where all five methods terminated optimally. This paired subset avoids comparing means computed from different sets of solved instances.

On the four paired optimal cells, the corrected legacy method has the lowest mean CPU and wall time; the ordered formulations do not show a timing advantage on this subset. Average model sizes are descriptive and depend on the chosen domain and maintenance-timing semantics. The original and split rows have lower objectives but violate the independent hour replay, so they are not feasible performance baselines.

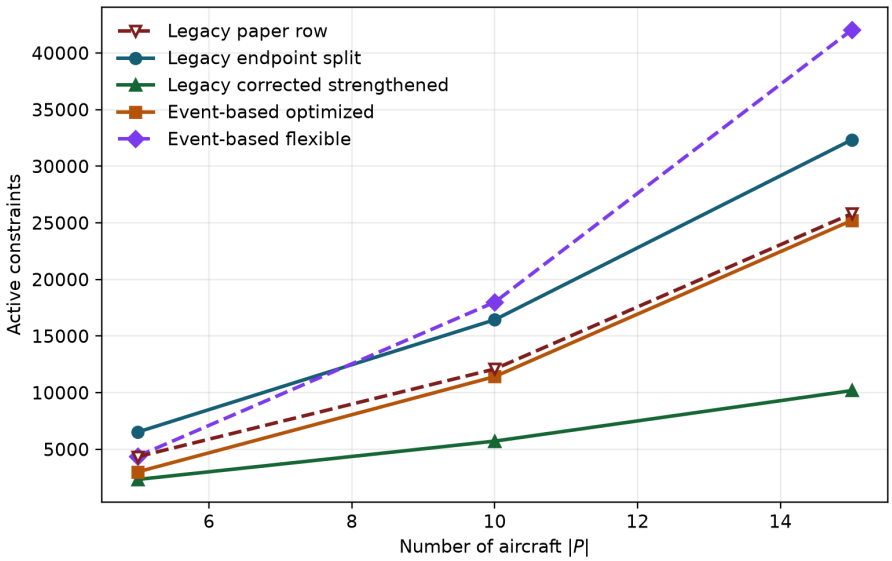


Figure 4: Active Pyomo constraints versus fleet size at observed horizon $H = 30$.

6.4.2. Cumulative-prefix LP and MILP ablation

We compared the compact strict interval formulation with an opt-in cumulative daily-prefix representation on the four P=5/10 instances. Both use the same strict C13/C13b conditions, zero-day legacy deferral, and bound-preserving LP relaxation. All interval and prefix LPs and MILPs terminated optimally on these four cases. The prefix projection reproduces the interval LP objective to numerical precision, as expected from the telescoping reformulation in Section 4.11, and the two formulations return identical integer objectives. The exact day-state model has a stronger LP bound on all four instances, so prefix-versus-interval equivalence should not be misread as equivalence to that stronger relaxation. The P=15 LP ablation is excluded: several solver runs reached their iteration limit and returned no reliable LP objective.

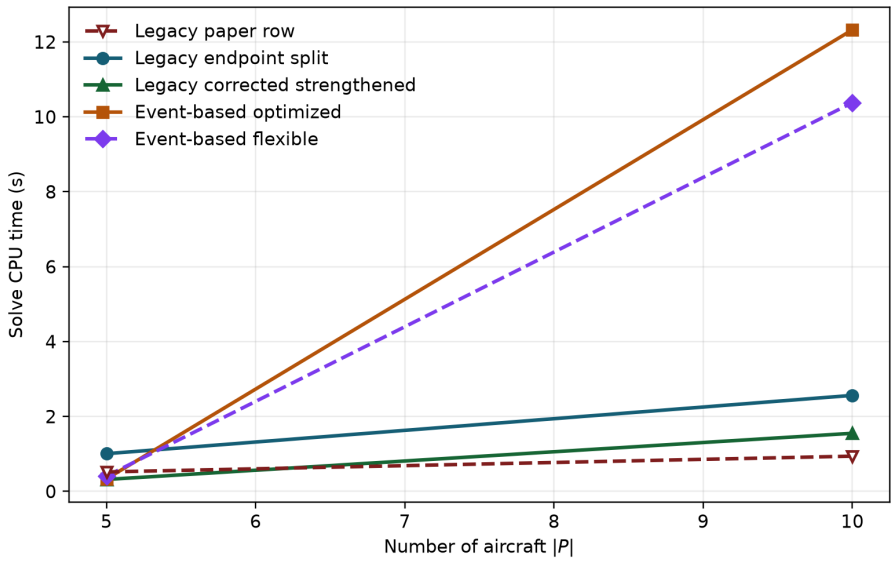


Figure 5: A-only solve CPU time versus fleet size at observed horizon $H = 30$. Only paired fleet sizes where all five methods are optimal are plotted.

The prefix representation adds continuous states and flow rows, and it does not improve solve time uniformly: it is faster than interval on two cells and slower on two. This is a formulation-equivalence and size observation for the tested A-only subset, not evidence of a general performance advantage. Raw LP/MILP rows and run metadata are stored in `results/tables/a_only_hour_prefix_lp_20261020.csv` and `results/tables/a_only_hour_pr`

6.4.3. Three-seed A-only replication subset

To measure instance-to-instance variation, we generated three deterministic replicates for each of the four P=5/10, H=30/48/49 cells in `data/a_only_multiseed_20261020/`. All 60 method-instance runs terminated optimally. The strengthened legacy and both Event-based objectives matched on every replicate. Table 8 reports the mean common objective and wall time as mean ± sample standard devi-

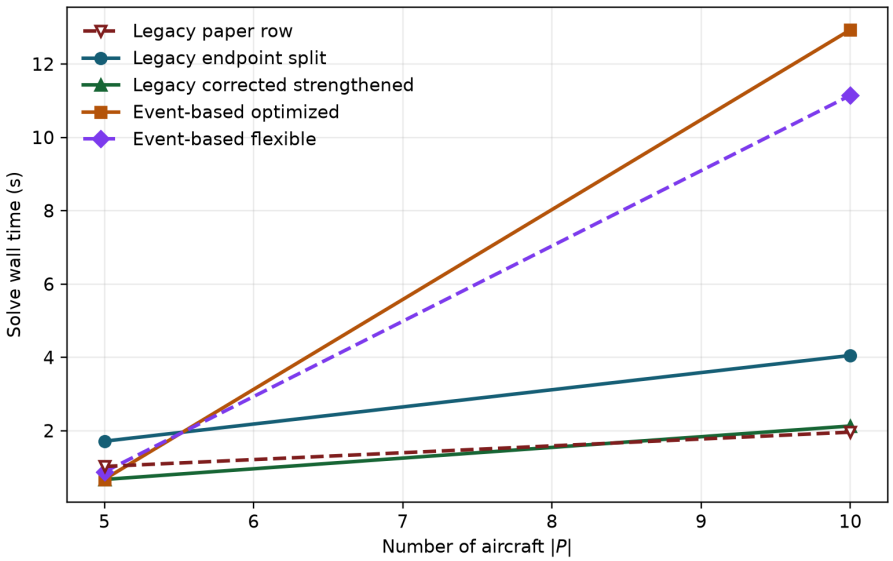


Figure 6: A-only solve wall time versus fleet size at observed horizon $H = 30$. Only paired fleet sizes where all five methods are optimal are plotted.

ation. These three replicates are a limited robustness check, not a basis for significance tests or a general runtime ranking.

The P=10, H=30 Event-based wall-time standard deviations are large relative to their means, illustrating why the single-instance scale-up timings should not be interpreted as stable method rankings. All five methods pass the objective/status checks in this subset, but the independent replay still rejects the paper-C13 and endpoint-split schedules; their lower objectives must not be treated as feasible comparisons. Raw rows and provenance are in `results/tables/a_only_multiseed_20261020.csv` and its metadata sidecar.

6.4.4. Conditional removal of unnecessary A checks

We also tested the sufficient pruning condition that removes optional A-check triggers when a tail's initial exposure plus all assignable flight hours is below the A threshold. On a modified six-flight A-only control, each tail had at most 20 assignable flight hours and zero initial exposure, versus a 100,000-minute threshold (1,666.7 hours). The unpruned and pruned models both solved optimally at objective 600 and passed independent replay. Pruning removed all 12 trigger variables and reduced the model from 88 to 76 variables and from 187 to 172 constraints. This is a construction/model-size check on a deliberately qualifying instance; it does not establish a runtime improvement on operationally binding cases. Table 9 summarizes the run; the run and transformed-input hashes are recorded in `results/tables/a_check_pruning_20261020.csv` and its metadata sidecar.

6.4.5. Independent schedule replay and scope

The generator replay certifies the designated rotations; the separate post-solve replay audits optimizer-selected schedules. For legacy models it resets the hour counter after the selected maintenance day; for ordered event models it resets after the selected trigger flight and independently checks event timing, flight blocking, and station capacity. This protocol detects violations in the paper and split C13 variants and returns zero violations for the strengthened exact-state and optimal Event-based schedules in this suite. It does not certify the four Event-based incumbents stopped by the time limit.

6.4.6. Event-model comparison

Optimized and flexible Event-based objectives match the strengthened legacy optimum on the four jointly optimal P=5/10 cells. At P=15, both event methods return the same incumbent as strengthened legacy, but neither proves optimality at either horizon. No objective-equivalence claim is made beyond these matched cases.

6.4.7. Flexible-start Event-based variant

The flexible-start implementation uses the integer-grid E14–E21 formulation with a one-day deferral window, as described in Section 5.8. Its model has more variables and constraints than the immediate-start event model in each generated performance cell. The objective comparison above is limited to solver results and does not replace independent schedule validation.

The scale-up experiment uses one deterministic seed per (P, H) cell, one maintenance hub, A checks only, and a single CPLEX configuration. It is a reproducible instance-family comparison, not broad statistical evidence. Multiple seeds, varied airport networks and station capacities, and solver configuration sensitivity remain important follow-up experiments.

7. Conclusions and Future Work

This paper revisited the compact Tail Assignment Problem formulation of Khaled et al. (2018) and made three focused contributions. First, we showed that the basic continuity and turn-time inequalities admit a simultaneous-departure corner case and closed it with explicit non-overlap constraints. Second, we proved that the paper's cumulative-flight-time row can relax a maintenance interval incorrectly and that the natural endpoint split is also

insufficient. We provided an exact cumulative-state correction and verified the failure against the production constraint objects. Third, we developed a day-free event-based reformulation for type-A maintenance and an ordered prefix-state implementation. The event formulation attaches maintenance to its triggering flight and propagates the flight-hour counter along the aircraft's selected route. The five-method, six-instance A-only rerun matches the legacy and event check deferral policies more closely by setting the day-indexed legacy deferral cap to zero. On the four P=5/10 cells where all three corrected methods prove optimality, strengthened legacy, optimized Event-based, and flexible Event-based objectives agree. At P=15, both Event-based methods reach the time limit; their incumbents match the strengthened legacy incumbent, but optimality is not established.

Independent post-solve replay reports no flight-hour violations for the six strengthened legacy optima and the four optimal schedules from each Event-based method. The original paper-C13 and endpoint-split formulations produce 663 and 183 violations, respectively, across the six instances. Their lower objectives therefore do not represent feasible performance advantages under the replayed A-check semantics. The replay also checks ordered-event timing, capacity, and flight blocking; time-limited Event-based schedules remain unvalidated.

The results do not show a general Event-based speedup. On the four common optimal cells, the strengthened legacy model has lower mean CPU and wall time than either Event-based method, while the per-cell and model-size results vary. An optional A-check-pruning reduction also preserves the optimum on one constructed instance where no tail can reach its hour threshold,

but this is model-size evidence rather than a runtime result on maintenance-critical cases. The six-cell scale-up uses one seed per cell; a separate three-seed P=5/10 subset confirms objective matching and shows substantial timing variation on the P=10, H=30 cell. This small replication is descriptive and does not establish statistical performance ranking or solver-independent dominance. All results use one CPLEX 12.6 configuration.

Reproducibility.. The project contains the model implementations, deterministic instance generator, benchmark scripts, result tables, provenance sidecars, and plotting scripts needed to reproduce the reported snapshots. The legacy, event, and ordered A-only builders remain separately selectable so comparisons can distinguish model semantics from implementation choices.

Future directions

- **Larger A-only benchmark families:** repeat the horizon and fleet-size experiments over multiple seeds and heterogeneous timetable structures, reporting solver variability and confidence intervals.
- **Solver and configuration robustness:** repeat the comparison with and without presolve, under different time limits and warm-start policies, and with an independently implemented MILP solver.
- **Integrated fleet assignment and routing:** extend the corrected compact model while retaining physically executable route constraints.
- **Robust and recoverable variants:** study uncertainty in flight times, disruptions, and maintenance availability under the exact state formulation.

- **Heuristic methods:** develop constructive and local-search methods for the corrected A-only model and compare them with the exact formulations on larger instances.

References

Barnhart, C., Belobaba, P., Odoni, A.R., 2003. Applications of operations research in the air transport industry. Transportation Science 37, 368–391.

Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleeting and routing. Transportation Science 32, 208–220. doi:10.1287/trsc.32.3.208.

Borndörfer, R., Dovica, I., Nowak, I., Schickinger, T., 2010. Robust tail assignment. Technical Report. Konrad-Zuse-Zentrum für Informationstechnik Berlin.

Clarke, L., Johnson, E., Nemhauser, G., Zhu, Z., 1997. The aircraft rotation problem. Annals of Operations Research 69, 33–46. doi:10.1023/A:1018945415148.

Clarke, L.W., Hane, C.A., Johnson, E.L., Nemhauser, G.L., 1996. Maintenance and crew considerations in fleet assignment. Transportation Science 30, 249–260.

Cordeau, J.F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. Transportation Science 35, 375–388.

Desaulniers, G., Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1997. Daily aircraft routing and scheduling. Management Science 43, 841–855.

Froyland, G., Maher, S.J., Wu, C.L., 2013. The recoverable robust tail assignment problem. Transportation Science 48, 351–372.

Gopalan, R., Talluri, K.T., 1998. Mathematical models in airline schedule planning: A survey. Annals of Operations Research 76, 155–185.

Grönkvist, M., 2005. The tail assignment problem. Ph.D. thesis. Chalmers, Göteborg University.

Hane, C.A., Barnhart, C., Johnson, E.L., Marsten, R.E., Nemhauser, G.L., Sigismondi, G., 1995. The fleet assignment problem: Solving a large-scale integer program. Mathematical Programming 70, 211–232.

Haouari, M., Shao, S., Sherali, H.D., 2012. A lifted compact formulation for the daily aircraft maintenance routing problem. Transportation Science 47, 508–525.

Khaled, O., Minoux, M., Mousseau, V., Michel, S., Ceugniet, X., 2018. A compact optimization model for the tail assignment problem. European Journal of Operational Research 264, 548–557. doi:10.1016/j.ejor.2017.06.045.

Klabjan, D., 2005. Large-scale models in the airline industry, in: Desaulniers, G., Desrosiers, J., Solomon, M.M. (Eds.), Column Generation. Springer US, pp. 163–195.

Lacasse-Guay, E., Desaulniers, G., Soumis, F., 2010. Aircraft routing under different business processes. Journal of Air Transport Management 16, 258–263.

Lavoie, S., Minoux, M., Odier, E., 1988. A new approach for crew pairing problems by column generation with an application to air transportation. European Journal of Operational Research 35, 45–58.

Levin, A., 1971. Scheduling and fleet routing models for transportation systems. Transportation Science 5, 232–255.

Liang, Z., Chaovalitwongse, W.A., 2012. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. Transportation Science 47, 493–507.

Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. Transportation Science 45, 109–120.

Mercier, A., Cordeau, J.F., Soumis, F., 2005. A computational study of Benders decomposition for the integrated aircraft routing and crew scheduling problem. Computers & Operations Research 32, 1451–1476. doi:10.1016/j.cor.2003.11.013.

Minoux, M., 1984. Column generation techniques in combinatorial optimization, a new application to crew pairing problems, in: Proceedings of the AGIFORS Symposium, Strasbourg, France.

Sarac, A., Batta, R., Rump, C.M., 2006. A branch-and-price approach for

operational aircraft maintenance routing. European Journal of Operational Research 175, 1850–1869.

Sherali, H.D., Bish, E.K., Zhu, X., 2006. Airline fleet assignment concepts, models, and algorithms. European Journal of Operational Research 172, 1–30.

Table 3: Coverage of the numbered C1–C13f requirements in the ordered A-only formulation.

Requirement	Day-indexed role	Ordered A-only counterpart
C1	Binary assignment domain	Binary domain on x_{rj}.
C2	Flight coverage	Coverage equation on x_{rj} (equivalently $x_{i_r j}$).
C3–C4	Airport continuity and turn time	Corrected equipment-flow balance on assignment variables.
C5	Pairwise non-overlap	Pairwise overlap inequalities on x.
C6–C7	Maintenance variables z, y	Event indicator z_{rj}; the daily indicator y is not a primitive variable.
C8	Maintenance blocks later flights	Timestamp blocking using $\delta_A + \Delta t$; the flexible variant blocks until the deferred completion.
C9	Maintenance requires assignment	The event-assignment inequality above.
C10	Maintenance capacity	Direct timestamp-overlap rows at event starts; exact grid-based overlap-indicator rows in the flexible variant.
C11	Aggregate events by aircraft and day	Integer event count $n_{jd} = \sum_{r:\, d(i_r)=d} z_{rj}$.
C12	Calendar-time spacing	Inactive for type-A-only maintenance in this paper.
C13	Paper cumulative-flight-time row	Replaced by the exact ordered prefix state.
C13a	Daily assigned flight time	Implicit in the ordered flight sequence.
C13b–C13f	Exact cumulative-state recursion and bound	Ordered prefix-state rows (41).

Table 4: Four-way numeric verification of the cumulative-flight-time flaw. The fixed counterexample has $T^A_{\max} = 600$, $M_{14} = 1200$, and LHS $\sum_{i \in F_{1,4}} x_{ij} \tilde{t}_i = 1200$. "Accepts" means every row of that formulation is satisfied; "rejects" means the violation is detected.

Scenario	Khaled (13)	Legacy split	Legacy corrected strengthened	Event-b
Both endpoints checked	rejects	accepts (wrong)	rejects	
Only end checked	accepts (wrong)	rejects	rejects	

Table 5: Five-method A-only results on the longer-horizon, higher-flight-count family with zero-day legacy check deferral. "Replay" is the independent flight-hour audit (PASS or violation count); time-limited rows are not optimality certificates. CPU and wall times are in seconds.

$\|P\|$	H	F	Method	Status	Replay	Objective	CPU	Wall	Vars	Cons.
5	30	74	Legacy paper	optimal	51	34883	0.52	1.02	1020	4348
5	30	74	Legacy split	optimal	17	36083	1.01	1.71	1020	6523
5	30	74	Legacy corrected	optimal	PASS	37583	0.32	0.67	1175	2338
5	30	74	Event optimized	optimal	PASS	37583	0.33	0.67	925	3007
5	30	74	Event flex	optimal	PASS	37583	0.41	0.88	1363	4393
5	48	80	Legacy paper	optimal	60	37707	0.49	1.36	1335	8385
5	48	80	Legacy split	optimal	15	39207	0.92	2.24	1335	14025
5	48	80	Legacy corrected	optimal	PASS	40707	0.28	0.67	1580	3310
5	48	80	Event optimized	optimal	PASS	40707	0.30	0.67	1000	3240
5	48	80	Event flex	optimal	PASS	40707	0.29	0.68	1360	4320
10	30	146	Legacy paper	optimal	106	68850	0.94	1.96	3120	12095
10	30	146	Legacy split	optimal	30	71150	2.56	4.05	3120	16445
10	30	146	Legacy corrected	optimal	PASS	74150	1.55	2.13	3430	5725
10	30	146	Event optimized	optimal	PASS	74150	12.32	12.93	3650	11425
10	30	146	Event flex	optimal	PASS	74150	10.38	11.15	5458	18000
10	49	160	Legacy paper	optimal	120	75391	3.29	5.36	3900	20308
10	49	160	Legacy split	optimal	30	78391	10.12	13.62	3900	32068
10	49	160	Legacy corrected	optimal	PASS	81391	0.77	1.53	4400	8048
10	49	160	Event optimized	optimal	PASS	81391	0.98	2.04	4000	12672
10	49	160	Event flex	optimal	PASS	81391	1.57	2.37	5663	18269
15	30	218	Legacy paper	optimal	158	102849	1.72	3.73	6300	25822
15	30	218	Legacy split	optimal	46	106249	5.68	8.62	6300	32347
15	30	218	Legacy corrected	optimal	PASS	110749	12.49	13.48	6765	10192
15	30	218	Event optimized	maxIterations	–	110749	180.41	181.50	8175	25222
15	30	218	Event flex	maxIterations	–	110749	180.50	182.08	12265	42053
15	49	240	Legacy paper	optimal	168	113169	4.25	8.57	7650	36633
15	49	240	Legacy split	optimal	45	117669	15.42	22.45	7650	54273
15	49	240	Legacy corrected	optimal	PASS	122169	24.10	25.59	8400	13863
15	49	240	Event optimized	maxIterations	–	122169	180.45	181.75	9000	28039
15	49	240	Event flex	maxIterations	–	122169	180.52	182.35	12807	42276

Table 6: Mean active scalar model size over six generated cases; mean build, CPU, and wall times over the four common optimal cells. Times are in seconds.

Method	Optimal/6	Mean vars/6	Mean cons./6	Mean build/4	Mean CPU/4	Mean wall/4
Legacy paper	6/6	3887.5	17931.8	1.327	1.310	2.426
Legacy split	6/6	3887.5	25946.8	1.412	3.654	5.404
Legacy corrected	6/6	4291.7	7246.0	0.181	0.730	1.248
Event optimized	4/6	4458.3	13934.2	0.325	3.486	4.080
Event flex	4/6	6486.0	21551.8	0.334	3.161	3.766

Table 7: A-only cumulative-prefix ablation on four tractable instances. LP columns report optimal lower bounds; the MIP objective is shared by exact day-state, strict interval, and prefix models. Prefix model size and MIP wall time are shown for comparison.

$\|P\|$	H	F	Exact-state LP	Interval LP	Prefix LP	MIP objective	Interval vars/rows	Prefix vars/rows	Prefix wall (s)
5	30	74	36516.33	34383.06	34383.06	37583	1020/4493	1180/4648	0.91
5	48	80	39557.00	37207.06	37207.06	40707	1335/8620	1585/8865	1.05
10	30	146	72050.00	67850.11	67850.11	74150	3120/12385	3440/12695	8.24
10	49	160	79057.67	74391.13	74391.13	81391	3900/20788	4410/21288	4.61

Table 8: Three-seed A-only subset. Each cell has three generated instances; all 60 runs are optimal. Objective means agree across the three corrected methods. Wall time is mean $\pm$ sample standard deviation in seconds.

$\|P\|$	H	F	Mean objective	Strengthened legacy	Event optimized	Event flex
5	30	74	37603.7	0.689 ± 0.008	0.700 ± 0.011	0.812 ± 0.040
5	48	80	40706.3	0.695 ± 0.004	0.642 ± 0.007	0.679 ± 0.008
10	30	146	74219.3	2.064 ± 0.131	13.099 ± 7.063	12.975 ± 2.532
10	49	160	81405.3	1.527 ± 0.055	1.843 ± 0.265	2.395 ± 0.050

Table 9: Conditional A-check pruning on a constructed qualifying instance.

Variant	Status	Objective	z vars	Vars	Cons.	Wall (s)	Replay
Baseline	optimal	600.0	12	88	187	0.544	PASS
Pruned	optimal	600.0	0	76	172	0.513	PASS